Case 4 of 5 | API: JSONPlaceholder | Tool: pytest + requests

Case 4: PUT — Update a Resource

Send an updated payload for post 1 and verify the server returns the updated data.

API Endpoint

```
PUT https://jsonplaceholder.typicode.com/posts/1
```

What You Will Learn

- HTTP PUT method (full update)
- Status code 200 OK on update
- Sending all fields in request body
- Asserting updated values in response

Study Plan

Step 1 — PUT vs PATCH
PUT replaces the entire resource. You must send all fields. PATCH only updates specific fields. For this case, use PUT.

Step 2 — Build the updated payload
Create a dict with all fields: id=1, title='Updated Title', body='Updated body', userId=1.

Step 3 — Use requests.put()
Call requests.put(url, json=payload). The URL must include the resource id: /posts/1.

Step 4 — Assert status 200
A successful PUT returns 200 OK. Assert response.status_code == 200.

Step 5 — Verify the updated title
Assert that response.json()['title'] matches the new title you sent in the payload.

Your Task

Write a pytest test that sends a PUT request to /posts/1.
Payload: { id: 1, title: 'Updated Title', body: 'Updated body', userId: 1 }
The test must verify:
1. Status code is 200
2. The response 'title' equals 'Updated Title'
3. The response 'id' is 1

Hint: requests.put(url, json=payload). The URL is /posts/1, not /posts.

Base URL: https://jsonplaceholder.typicode.com | No authentication required | Free public API